

Code-Mix Sentiment Analyzer

A Comparative Study of Classical and Neural NLP Architectures on Bangla–English Code-Switched Text

NLP Laboratory Final Project Proposal — Revised Proposal

Table of Contents

- [Abstract](#)
- [1. Project Type](#)
- [2. Introduction](#)
- [3. Project Objectives](#)
- [4. Scope and Constraints](#)
- [5. Dataset Construction](#)
- [6. Methodology and Techniques Covered](#)
- [7. Model Architectures](#)
- [8. User Interface](#)
- [9. Evaluation Plan](#)
- [10. Expected Deliverables](#)
- [11. Tentative Timeline](#)
- [12. Tools and Technologies](#)
- [13. Conclusion](#)

Abstract

This proposal presents a final project for the NLP laboratory course, centered on sentiment classification of everyday, universally familiar text — movie reviews, food/restaurant reviews, and general social comments — expressed in English, Bangla, Banglish (Bangla written in Roman script), and natural Bangla–English code-switched form. Rather than restricting the project to a single family of models, the project is designed to exercise the full pipeline of techniques covered in the course: N-Gram and Bag-of-Words statistical baselines, TF-IDF feature representations, Word2Vec embeddings, classical ANN/RNN/LSTM/Attention/Transformer architectures implemented from scratch in PyTorch, and a fine-tuned multilingual BERT model used as a pretrained upper-bound benchmark. The project also includes a lightweight sequence-generation extension (next-word/next-sentence completion) to touch on autoregressive generation. All models are evaluated on an identical, held-out multilingual test set, and results are compared through a common, easy-to-read metric table and chart. A key deliverable is an interactive graphical user interface, built with Streamlit, that lets a user type or select a sentence in any of the four language forms and see live predictions and confidence scores from every trained model side by side.

1. Project Type

This is an implementation-oriented final project for the NLP laboratory course. It requires no novel research contribution; instead, it is scoped to demonstrate practical, hands-on mastery of the techniques taught across the semester — N-Gram models, Bag-of-Words, TF-IDF, Word2Vec, BERT, ANN, RNN, LSTM, Attention, and Transformer architectures — applied to classification, and, in a secondary capacity, sequence generation. The project additionally satisfies the requirement for a working, presentable user interface built using a Python UI library.

2. Introduction

In Bangladesh, everyday communication — whether in casual conversation, social media comments, or online reviews — frequently mixes Bangla and English within the same sentence or utterance, a phenomenon known as code-switching. Sentiment analysis is one of the most widely taught and immediately understandable NLP tasks: given a short piece of text, determine whether the

opinion expressed is positive, negative, or neutral. Because sentiment is something every reader can verify intuitively just by reading a sentence, it satisfies the requirement that the underlying data and task be accessible and familiar to any evaluator, without needing specialized background knowledge.

This project uses short, everyday reviews and comments — on movies, food, and general everyday experiences — as the corpus, collected and adapted into four parallel linguistic forms (English, Bangla, Banglish, and code-switched Bangla–English). The project implements and compares the complete spectrum of techniques taught in the course, from simple frequency-based statistical methods to deep contextual embeddings from a pretrained multilingual transformer, in order to observe how each technique's performance is affected as input shifts from single-language to code-switched text.

3. Project Objectives

The specific objectives of the project are as follows:

- To build a small, natural-language corpus of sentiment-labeled reviews/comments in English, Bangla, Banglish, and Bangla–English code-switched form, using topics (movies, food, everyday experiences) that require no specialized domain knowledge to understand.
- To implement classical statistical baselines — N-Gram language features, Bag-of-Words, and TF-IDF — paired with simple classifiers (e.g., Naive Bayes / Logistic Regression) for sentiment classification.
- To train Word2Vec embeddings on the project corpus and compare them against randomly initialized embeddings within the neural models.
- To implement an ANN, an RNN, an LSTM (with an attention mechanism), and a Transformer encoder from scratch in PyTorch for sentiment classification.
- To fine-tune a pretrained multilingual BERT model (e.g., mBERT or XLM-R) as an upper-bound benchmark against the from-scratch models.
- To extend one of the sequence models (LSTM or Transformer) into a small autoregressive next-word/next-sentence completion demo, to additionally demonstrate sequence-generation capability.
- To evaluate all models on an identical held-out test set, broken down by language condition (English, Bangla, Banglish, code-switched), to measure how robust each technique is to code-switching.
- To build an interactive Streamlit-based user interface allowing live text input, model selection, and side-by-side comparison of predictions.

4. Scope and Constraints

The following constraints define the scope of the project:

- **Framework:** All neural models are implemented and trained in PyTorch. TensorFlow is not used anywhere in the project.
- **Familiar, general-purpose corpus:** The dataset is restricted to universally relatable topics (movies, food, everyday experiences) so that any reader can understand and sanity-check example sentences without specialized knowledge.
- **Classical and neural methods both included:** Unlike a purely from-scratch, no-pretrained-anything approach, this project deliberately includes N-Gram, Bag-of-Words, TF-IDF, Word2Vec, and a fine-tuned multilingual BERT model, since these are core techniques taught in the course and are meant to be demonstrated, not avoided.
- **From-scratch neural core:** The ANN, RNN, LSTM, Attention, and Transformer models are implemented from first principles in PyTorch (no pretrained weights for these specific models), so that the course's architectural concepts are demonstrated directly.
- **Custom tokenizer for classical/from-scratch models:** A tokenizer for the mixed Bangla–English vocabulary is implemented manually for the classical baselines and from-scratch neural models. The pretrained BERT benchmark uses its own native subword tokenizer, since that is intrinsic to how pretrained transformers operate.
- **Small but natural dataset:** Given limited time and compute, the dataset is kept modest in size, but sentences are written/adapted to sound like natural reviews and comments rather than rigid templated statements.
- **Mandatory user interface:** A working graphical interface (Streamlit) is a required deliverable, not optional.

5. Dataset Construction

The dataset consists of short sentiment-labeled reviews and comments on familiar, everyday topics — movies, food and restaurants, and general everyday experiences/products. Each example is written or adapted in four parallel linguistic forms:

- **English** — e.g., "The movie's story was really boring and too long."
- **Bangla** — the same review expressed fully in the Bangla script.
- **Banglish** — the same review expressed in Bangla, written using the Roman (English) alphabet.
- **Code-switched** — the same review naturally mixing Bangla and English within a single sentence, similar to how people actually type in everyday chat and comments.

The corpus is organized into three sentiment classes: **Positive**, **Negative**, and **Neutral**. To build it:

- A base set of natural sentences is written and/or lightly adapted from publicly visible, non-sensitive review-style text (e.g., generic movie/food opinions), then manually translated and transliterated into the four language forms to keep meaning and label consistent across forms.
- A large language model may optionally be used only as an auxiliary drafting aid to help generate varied phrasings for the human-reviewed dataset — not as a component whose performance is measured in the evaluation.
- Each sentence is manually reviewed for label correctness and natural phrasing before being added to the dataset.
- The dataset is split into training, validation, and test sets, with the test set stratified by both sentiment label and language condition, so that per-language performance can be measured fairly for every model.

6. Methodology and Techniques Covered

The project is structured so that every major technique taught in the course is represented in the pipeline, rather than concentrating only on from-scratch neural models:

Stage	Techniques Used
Preprocessing	NLTK tokenization/normalization for English; manually implemented tokenizer for Bangla and Banglish/code-switched text; stopwords handling and basic text cleaning.
Classical baselines	N-Gram feature extraction; Bag-of-Words representation; TF-IDF weighting, each paired with a simple classifier (Naive Bayes and/or Logistic Regression) as statistical baselines.
Word embeddings	Word2Vec (Skip-gram/CBOW) trained on the project corpus; compared against randomly initialized embeddings learned end-to-end inside the neural models.
From-scratch neural models	ANN (bag-of-words/fixed-window input), RNN, LSTM, LSTM+Attention, and a small Transformer encoder — all implemented and trained from random initialization in PyTorch.
Pretrained benchmark	Fine-tuning a pretrained multilingual BERT model (mBERT or XLM-R) on the same classification task, used as an upper-bound reference point.
Sequence generation (bonus)	A small autoregressive next-word/next-sentence completion demo built on the trained LSTM or Transformer, to additionally illustrate sequence-generation behavior.
Interface	A Streamlit web app for live text input, model selection, and comparative visualization of predictions.

7. Model Architectures

All models are trained and evaluated on identical data splits so that the resulting comparison is fair and controlled.

7.1 Classical Statistical Baselines

- **N-Gram + classifier:** unigram/bigram frequency features feeding a simple classifier, serving as the simplest baseline.
- **Bag-of-Words + classifier:** sparse word-count vectors feeding a simple classifier.
- **TF-IDF + classifier:** term-frequency/inverse-document-frequency weighted vectors feeding a simple classifier, expected to outperform raw Bag-of-Words.

7.2 From-Scratch Neural Architectures (PyTorch)

- **ANN (Baseline):** a feed-forward network over Bag-of-Words/TF-IDF or averaged Word2Vec input, expected to perform worse on tasks needing word order or long-range dependency.
- **RNN:** a basic recurrent network processing tokens sequentially, maintaining a hidden state across time steps.
- **LSTM:** used to evaluate whether improved long-range memory retention helps on code-switched input.
- **LSTM/RNN + Attention:** a manually implemented attention mechanism layered on the recurrent model, allowing the model to selectively weight relevant parts of the sequence.
- **Transformer Encoder:** a small self-attention-based encoder with positional encodings, implemented entirely from scratch, used for the final classification layer.

7.3 Pretrained Benchmark

- **Multilingual BERT** (mBERT or XLM-R), fine-tuned on the project's classification data, included specifically to demonstrate contextual pretrained embeddings and to give a realistic upper bound against which the from-scratch models are compared.

8. User Interface

A key deliverable of the project is a working, presentable graphical interface, built using **Streamlit** (a pure-Python library for building clean interactive web apps quickly). The interface allows a user to:

- Type or paste a sentence in English, Bangla, Banglish, or code-switched form into a text box.
- Select which model(s) to run — N-Gram, Bag-of-Words, TF-IDF, ANN, RNN, LSTM, Attention model, Transformer, or the fine-tuned BERT benchmark.
- View the predicted sentiment label and confidence score for each selected model, displayed side by side.
- View a simple bar chart (Matplotlib, embedded in the Streamlit app) comparing all models' confidence scores for the same input sentence.
- Optionally browse a few sample sentences from the test set, organized by language condition, to quickly explore how each model behaves across English, Bangla, Banglish, and code-switched input.

Gradio is considered as a fallback alternative if any Streamlit-specific limitation arises during development, since both are pure-Python and require no separate frontend code.

9. Evaluation Plan

- Each model (classical and neural) is trained and evaluated using identical training, validation, and test splits.
- Accuracy, precision, recall, and F1-score are measured separately for English, Bangla, Banglish, and code-switched test examples.
- Results are compared across all models — N-Gram, Bag-of-Words, TF-IDF, ANN, RNN, LSTM, Attention model, Transformer, and fine-tuned BERT — to identify which technique is most robust to code-switched input.
- A brief qualitative error analysis presents representative misclassified examples for each model, with short explanations of likely causes (e.g., unseen code-switch patterns, tokenization errors, class imbalance).
- For the sequence-generation bonus component, a handful of generated completions are shown qualitatively rather than scored numerically, since generation quality assessment is outside the project's primary evaluation scope.

10. Expected Deliverables

- A small, natural-language, human-reviewed Bangla–English sentiment dataset (movies/food/everyday topics) in four parallel language forms, including the scripts used to assemble and split it.
- Implementations of N-Gram, Bag-of-Words, and TF-IDF baselines with a simple classifier.
- A Word2Vec embedding model trained on the project corpus.
- PyTorch implementations of the ANN, RNN, LSTM, Attention-augmented, and Transformer models, built entirely from scratch.
- A fine-tuned multilingual BERT model used as a benchmark.
- A small autoregressive text-completion demo built on the trained LSTM/Transformer.
- Training and evaluation code, with results logged for every model and every language condition.
- A comparative results table and accompanying chart summarizing accuracy/F1 across all models and language conditions.

- An interactive Streamlit user interface for live testing and model comparison.
- A final written report summarizing project setup, methodology, results, and observations.

11. Tentative Timeline

Week(s)	Activity
Week 1	Corpus collection/writing, translation into Bangla/Banglish/code-switched forms, manual labeling and review; custom tokenizer implementation.
Week 2	N-Gram, Bag-of-Words, and TF-IDF baselines with a simple classifier; Word2Vec training on the corpus.
Week 3	Implementation and training of the ANN and RNN models from scratch in PyTorch.
Week 4	Implementation and training of the LSTM and Attention-augmented models.
Week 5	Implementation and training of the Transformer encoder; fine-tuning the multilingual BERT benchmark.
Week 6	Evaluation, comparative and error analysis; sequence-generation demo; Streamlit UI development; final report preparation.

12. Tools and Technologies

- **Framework:** PyTorch for all neural model implementation and training (no TensorFlow).
- **Programming Language:** Python.
- **NLP Libraries:** NLTK for tokenization/preprocessing utilities; Gensim (or equivalent) for Word2Vec training; Hugging Face Transformers for the pretrained multilingual BERT benchmark.
- **Data Handling:** Custom dataset assembly scripts and a manually implemented tokenizer for Bangla/Banglish/code-switched text.
- **User Interface:** Streamlit (with Gradio as a fallback) for the interactive web-based demo.
- **Evaluation and Visualization:** Standard classification metrics (accuracy, precision, recall, F1), visualized using Matplotlib.

13. Conclusion

This revised project provides a structured, hands-on demonstration of the complete range of NLP techniques taught in the course — from N-Gram, Bag-of-Words, and TF-IDF statistical baselines, through Word2Vec embeddings, to from-scratch ANN, RNN, LSTM, Attention, and Transformer architectures, and finally a fine-tuned pretrained multilingual BERT benchmark — applied to a universally familiar task (sentiment analysis of everyday movie, food, and general comments) and evaluated specifically on Bangla–English code-switching. Paired with an interactive Streamlit interface for live demonstration, the project satisfies the corpus-familiarity requirement, the user-interface requirement, and gives a comprehensive, course-aligned comparison of how classical and neural NLP techniques handle linguistically mixed input.